Studio Nx

백엔드 240 — 이펙트 API

2026-10-18

Backend(FastAPI/Python) 240 — 이펙트 API (목록 · 붙이기 · 고치기 · 떼기)

항목	내용
파일	`studionx/backend/app/api/v1/tracks.py` (이펙트 경로)
작업	S1-09 기본 악기·이펙트: 트랙 안 이펙트 칸 경로 네 개 (PRD 02 FR-245, FR-263, FR-264)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	239 악기 API · 241 연결선 API

1. 하는 일

트랙의 이펙트 체인을 다루는 경로 네 개를 엽니다.

경로	하는 일	상태 코드
`GET /tracks/{track_id}/plugins`	이펙트 목록 (칸 번호 순 = 체인 순서)	200, 401, 404
`POST /tracks/{track_id}/plugins`	이펙트 붙이기	201, 400, 401, 403, 404
`PATCH /tracks/{track_id}/plugins/{track_plugin_id}`	순서·카드 자리·파라미터·바이패스 고치기	200, 400, 401, 403, 404
`DELETE /tracks/{track_id}/plugins/{track_plugin_id}`	이펙트 떼기	204, 401, 403, 404

주소 이름이 `/plugins` 인 이유는 화면이 부르는 이름(이펙트 체인)을 그대로 쓰기 위해서입니다. 표 이름(`track_plugins`)과 경로 번호 이름(`track_plugin_id`)은 서비스·DB 와 맞춰 두었습니다.

2. 언제 쓰나

쓰는 곳	
화면의 [Effects] 칸	체인 카드를 그릴 때 `GET`, 새 이펙트를 붙일 때 `POST`
카드 끌어 순서 바꾸기	옮긴 카드의 새 칸 번호를 `PATCH` 로 보냅니다 (FR-263)
바이패스 단추	`PATCH` 로 `is_bypassed` 만 보냅니다
갈래(패처) 화면	카드를 옮긴 자리를 `PATCH` 로 `node_x`·`node_y` 에 저장합니다 (FR-269)

3. 코드

목록과 붙이기

```
@router.get("/tracks/{track_id}/plugins", response_model=list[TrackPluginResponse],
            responses={...})
def list_effects(track_id: Annotated[int, Path(ge=1)], db: Db, current_user: CurrentUser):
    effects = track_service.list_effects(db, current_user, track_id)
    return [TrackPluginResponse.model_validate(p) for p in effects]
```

```
@router.post("/tracks/{track_id}/plugins", response_model=TrackPluginResponse,
             status_code=status.HTTP_201_CREATED, responses=WRITE_RESPONSES)
def add_effect(track_id: Annotated[int, Path(ge=1)], req: TrackPluginCreateRequest,
               db: Db, current_user: CurrentUser):
    tp = track_service.add_effect(
        db, current_user, track_id,
        plugin_id=req.plugin_id, slot_index=req.slot_index, node_x=req.node_x, node_y=req.node_y,
        params=req.params, is_bypassed=req.is_bypassed,
    )
    return TrackPluginResponse.model_validate(tp)
```

고치기와 떼기

```
@router.patch("/tracks/{track_id}/plugins/{track_plugin_id}",
              response_model=TrackPluginResponse, responses=WRITE_RESPONSES)
def update_effect(track_id: Annotated[int, Path(ge=1)],
                  track_plugin_id: Annotated[int, Path(ge=1)],
                  req: TrackPluginUpdateRequest, db: Db, current_user: CurrentUser):
    data = req.model_dump(exclude_unset=True)
    tp = track_service.update_effect(db, current_user, track_id, track_plugin_id, **data)
    return TrackPluginResponse.model_validate(tp)


@router.delete("/tracks/{track_id}/plugins/{track_plugin_id}",
               status_code=status.HTTP_204_NO_CONTENT, responses={...})
def remove_effect(track_id: Annotated[int, Path(ge=1)],
                  track_plugin_id: Annotated[int, Path(ge=1)],
                  db: Db, current_user: CurrentUser):
    track_service.remove_effect(db, current_user, track_id, track_plugin_id)
    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

문법 풀이

식	뜻
`{track_plugin_id}`	카드 한 장의 행 번호입니다. 235 의 `_require_effect` 가 트랙과 맞는지 확인합니다
`req: TrackPluginCreateRequest`	`slot_index` 가 반드시 있어야 합니다(기본값 없음). 없으면 422 입니다
`exclude_unset=True`	순서만 바꿀 때 파라미터가 빈 사전으로 덮이지 않게 합니다
응답을 카드 한 장으로	붙이기·고치기는 바뀐 카드를 그대로 돌려줍니다. 화면이 카드만 다시 그리면 됩니다
목록 응답이 배열	연결선·악기 목록과 같은 모양입니다. 개수는 화면이 `length` 로 셉니다

4. 왜 이렇게

선택	이유
순서 바꾸기를 `PATCH` 로	옮긴 카드 한 장만 보내면 됩니다. 목록 전체를 다시 보내는 방식은 동시 편집에서 서로 덮어씁니다
이미 찬 칸이면 400	같은 칸 번호가 둘이면 소리 순서를 정할 수 없습니다. 서비스가 DB UNIQUE 전에 확인합니다 (235)

선택	이유
카드 자리를 같은 경로에서 수정	자리와 순서는 별개입니다. 자리만 바꾸면 소리 순서는 그대로입니다 (FR-269)
이펙트 플러그인만 허용	악기를 체인에 넣으면 신호 흐름이 끊깁니다. 400 으로 알려 줍니다 (FR-245)

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 붙이기 201과 응답 칸 이름, `node_x` 저장, 기본값(`is_bypassed` = false)
- 악기 플러그인을 넣으면 400, 없는 플러그인 404
- 목록 개수, 바이패스 수정, 칸 번호 2로 옮기기, 남이 쓰는 칸으로 옮기면 400
- 떼기 204